

Multimodal Visual Understanding + SLAM Navigation

Multimodal Visual Understanding + SLAM Navigation

1. Course Content
2. Starting the Agent
 - 2.1 Configuring the Map Mapping File
 - 2.2 Configuring Map Mapping Variables in Dify
3. Running Examples
 - 3.1 Starting the Program
 - 3.2 Test Cases
4. Source Code Analysis

1. Course Content

- Basic: Run the example program to perform a comprehensive task using the robot's visual understanding capabilities combined with SLAM navigation.
- Advanced: Master the key source code introduced in this section.

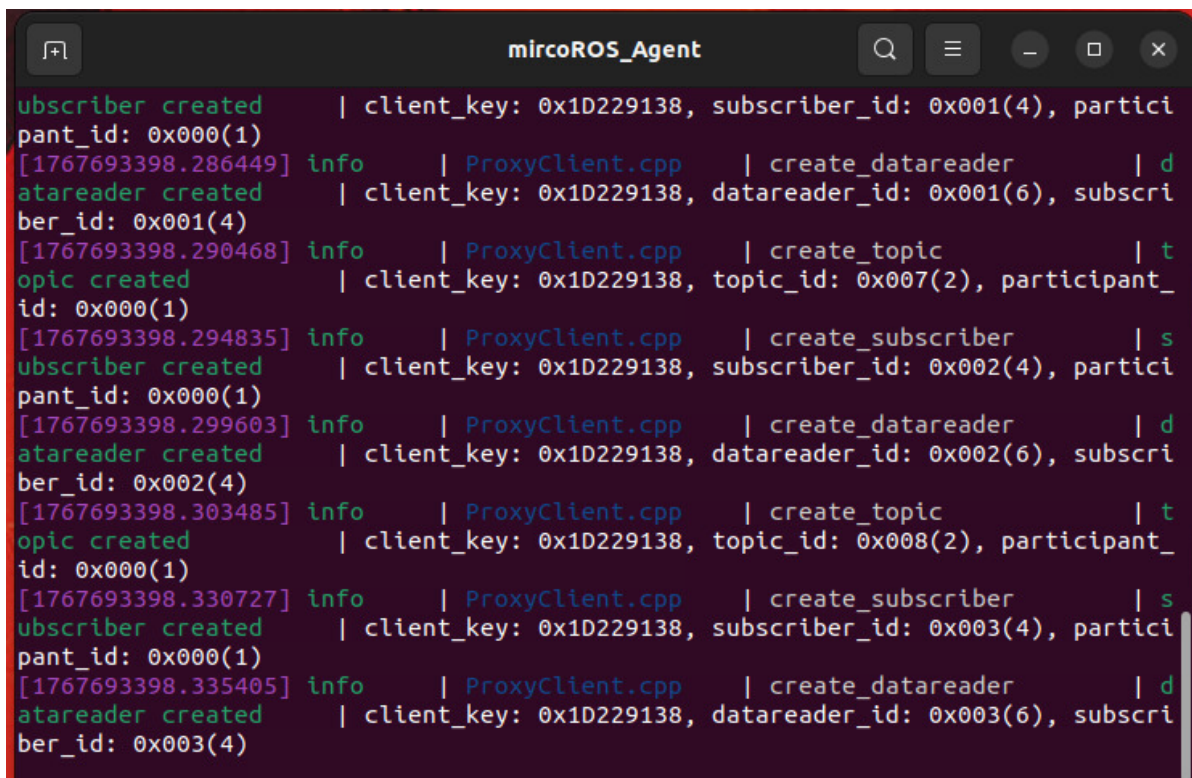
2. Starting the Agent

Note: If it has already been started, there is no need to start it again.

Enter the command on the vehicle terminal:

```
start_agent
```

The terminal prints the following information, indicating a successful connection.



```
subscriber created | client_key: 0x1D229138, subscriber_id: 0x001(4), partici
pant_id: 0x000(1)
[1767693398.286449] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x001(6), subscri
ber_id: 0x001(4)
[1767693398.290468] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x007(2), participant_
id: 0x000(1)
[1767693398.294835] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x1D229138, subscriber_id: 0x002(4), partici
pant_id: 0x000(1)
[1767693398.299603] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x002(6), subscri
ber_id: 0x002(4)
[1767693398.303485] info | ProxyClient.cpp | create_topic | t
opic created | client_key: 0x1D229138, topic_id: 0x008(2), participant_
id: 0x000(1)
[1767693398.330727] info | ProxyClient.cpp | create_subscriber | s
ubscriber created | client_key: 0x1D229138, subscriber_id: 0x003(4), partici
pant_id: 0x000(1)
[1767693398.335405] info | ProxyClient.cpp | create_datareader | d
atareader created | client_key: 0x1D229138, datareader_id: 0x003(6), subscri
ber_id: 0x003(4)
```

[!NOTE]

Note: To experience this course, you need to first build at least one grid map according to the LiDAR chapter.

2.1 Configuring the Map Mapping File

- Command to start navigation on the vehicle's infotainment system

For Raspberry Pi 5 users, if the ROS container is not running, you need to start it first.

```
bringup_m3
```

Open a terminal inside the container.

```
exec_m3
```

(ORIN mainboard)

```
ros2 launch m3_bringup nav2.launch.py
```

(RDK & PI5 mainboard) Step-by-step execution, visualized on the accompanying virtual machine Rosmaster-M3-Ubuntu22.04, start it.

```
#Virtual machine side
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz

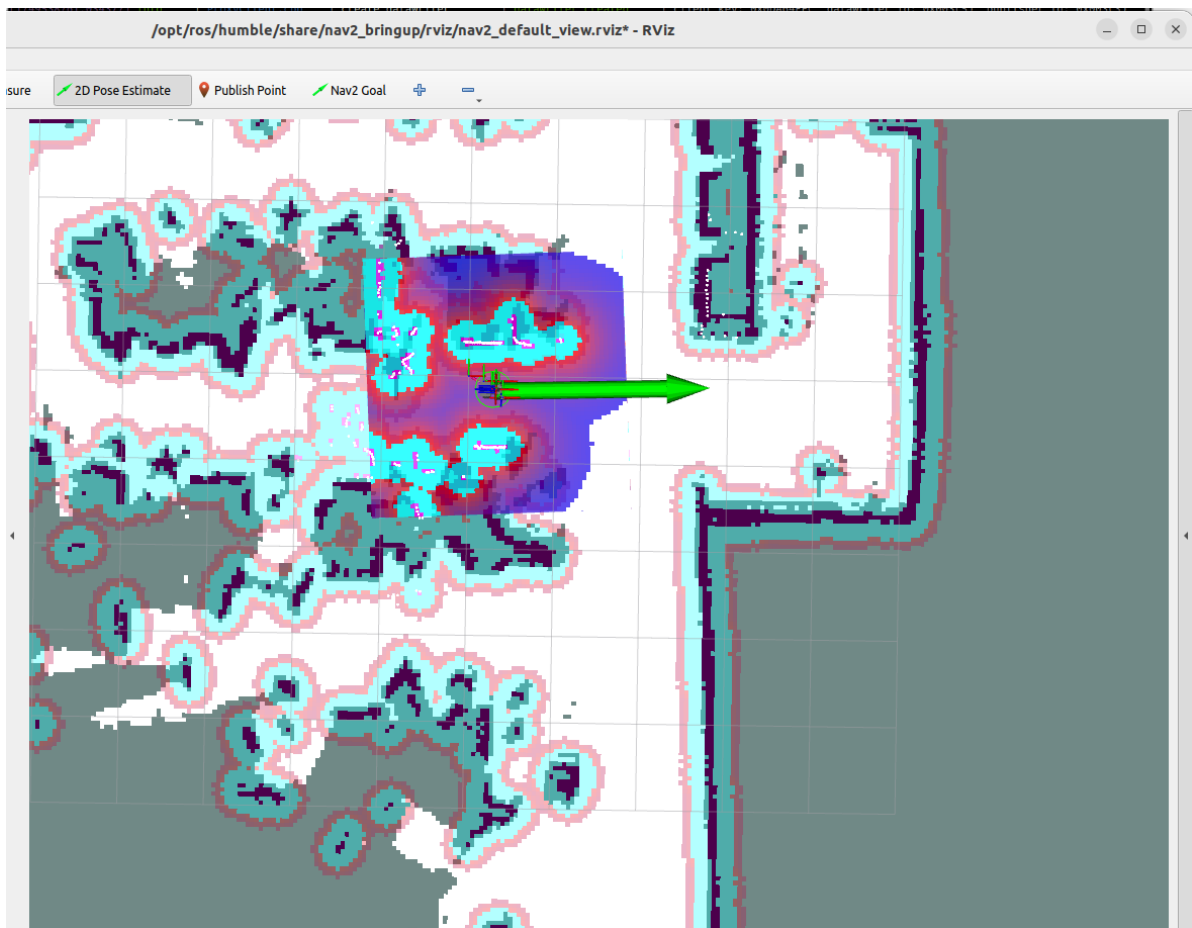
#Host side
ros2 launch m3_bringup nav2.launch.py gui:=False
```

[!TIP]

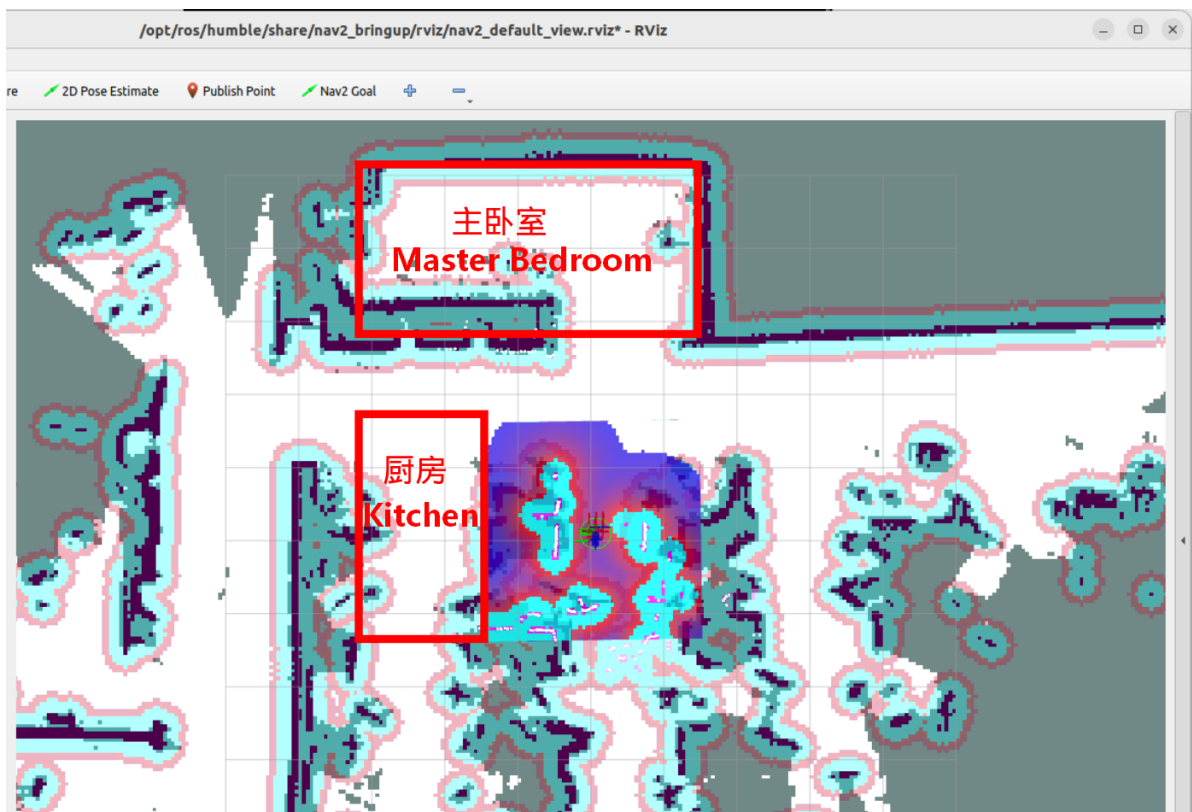
- The AMCL positioning algorithm is used for navigation by default here, which is suitable for most applicable scenarios. If higher positioning accuracy is required, please refer to the product tutorial [02.Lidar Course]—[13.slam_toolbox localization and navigation] section to build a pose map and use the slam_toolbox pure positioning mode for navigation.

Next, the rviz2 visualization interface will open. Click on **2D Pose Estimate** in the top toolbar to enter selection mode. Roughly mark the robot's location and orientation on the map.

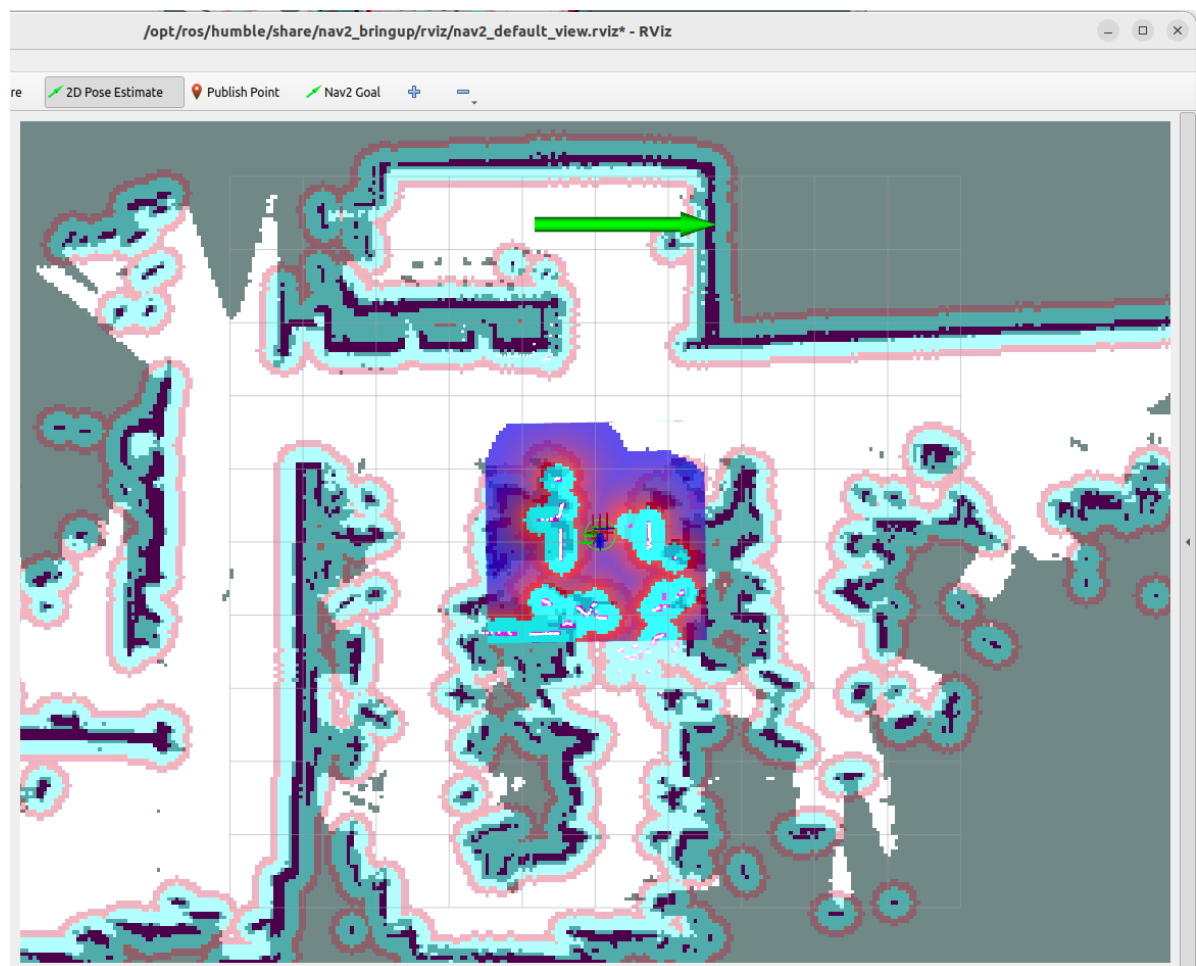
The robot's model will be displayed on the map, as shown below:



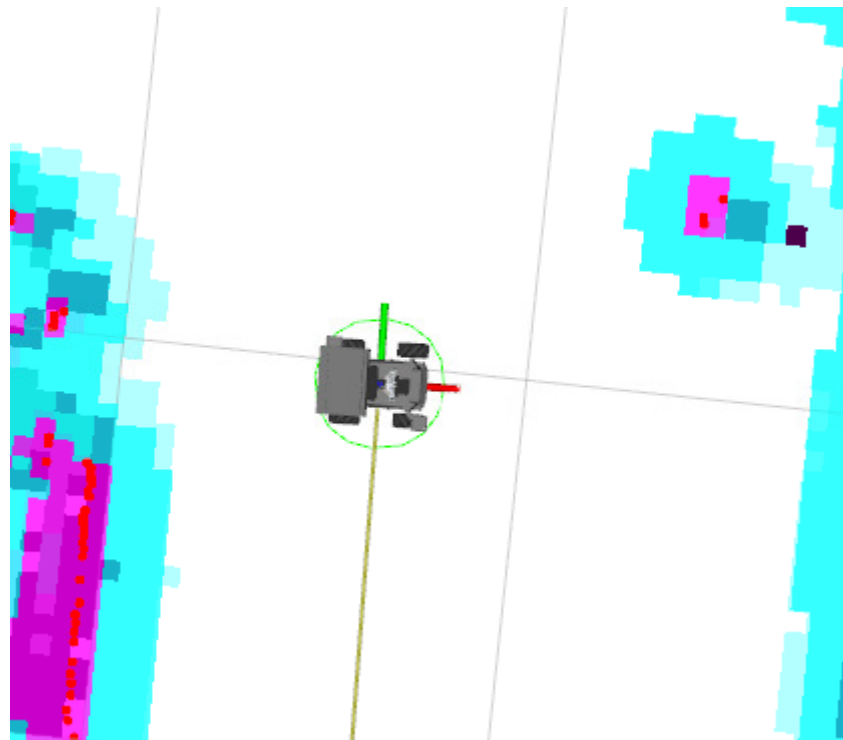
We can name any precise point on the map. Here, we use "Master Bedroom" and "Kitchen" as examples.



As shown in the image below, we first click on **Nav2** The Goal tool guides the vehicle's navigation system to the target location we need to mark.



Wait for the vehicle's navigation system to reach the target location to be marked.



- Run the following command in the terminal to obtain the robot's pose information in the map coordinate system:

```
ros2 run tf2_ros tf2_echo map base_footprint
```

```

jetson@yahboom:~$ ros2 run tf2_ros tf2_echo map base_footprint
[INFO] [1754377006.239762954] [tf2_echo]: Waiting for transform map -> base_footprint: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
[INFO] [1754377008.204300151] [tf2_echo]: Waiting for transform map -> base_footprint: Invalid frame ID "map" passed to canTransform argument target_frame - frame does not exist
At time 1754377008.700000000
- Translation: [3.974, -2.634, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.688, 0.726]
- Rotation: in RPY (radian) [0.000, 0.000, -1.516]
- Rotation: in RPY (degree) [0.000, 0.000, -86.867]
- Matrix:
  0.055  0.999  0.000  3.974
 -0.999  0.055 -0.000 -2.634
 -0.000  0.000  1.000  0.000
  0.000  0.000  0.000  1.000
At time 1754377009.734000000
- Translation: [3.974, -2.634, 0.000]
- Rotation: in Quaternion [0.000, 0.000, -0.688, 0.726]
- Rotation: in RPY (radian) [0.000, 0.000, -1.517]
- Rotation: in RPY (degree) [0.000, 0.000, -86.943]
- Matrix:
  0.053  0.999  0.000  3.974
 -0.999  0.053 -0.000 -2.634
 -0.000  0.000  1.000  0.000

```

Open the `map_mapping.yaml` map mapping file (you can choose any method to open it, such as VNC, vscode, or command line):

- Here's an example of opening the map configuration file via command line:

```
nano ~/yahboomM3_ws/user_ws/src/m3_bringup/config/multi_brains/map_mapping.yaml
```

```

GNU nano 6.2 /home/jetson/M3Pro_ws/multi_brains_file/map_mapping.yaml
#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure
#地图映射Map mapping

common_map_areas: #常规导航 common navigation
A:
  name: 'xxx'
  position:
    x: 1.824
    y: -0.838
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: -0.663
    w: 0.748
B:
  name: 'xxx'
  position:
    x: 1.488
    y: 0.661
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.725
    w: 0.688

```

- Modify the symbolic pose under the `common_map_areas` field, where `name` is the location name, and fill in the previously obtained poses into the `position` and `orientation` fields.

```

#根据实际的场景环境，自定义地图中的区域，可以添加任意个区域，注意和大模型的地图映射保持一致即可
#According to the actual scene environment, customize the areas in the map. You can add any number of areas, just make sure they are consistent with the map mapping of the large model
#地图映射Map mapping

```

```

common_map_areas: #常规导航 common navigation
A:
  name: 'Master bedroom'
  position:
    x: 3.974
    y: -2.634
  orientation:
    x: 0.0
    y: 0.0
    z: -0.688

```

```

w: 0.726
B:
  name: 'xxx'
  position:
    x: 1.488
    y: 0.661
    z: 0.0
  orientation:
    x: 0.0
    y: 0.0
    z: 0.725
    w: 0.688

```

- After configuring all the points that need to be marked, save the file (Ctrl+X) for immediate effect.

2.2 Configuring Map Mapping Variables in Dify

- After configuring the map mapping file above, we also need to ensure that the AI model knows in advance the relationships between locations and symbols in these maps.
- Start the Dify service (if it is already running, there is no need to start it again):

```
bringup_dify
```

```

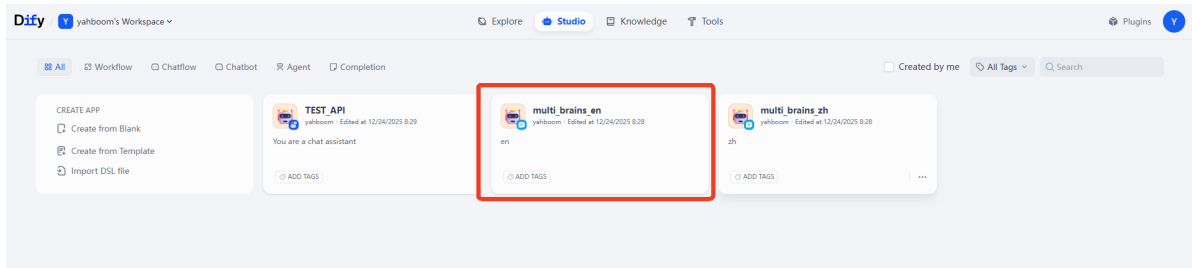
pi@raspberrypi: ~
pi@raspberrypi: ~ 80x24
pi@raspberrypi:~ $ bringup_dify
[+] up 14/14
✓ Network docker_default Created 0.0s
✓ Network docker_ssrf_proxy_network Created 0.1s
✓ Container docker-init_permissions-1 Exited 7.8s
✓ Container docker-sandbox-1 Created 2.9s
✓ Container docker-web-1 Created 2.9s
✓ Container docker-redis-1 Created 2.9s
✓ Container docker-weaviate-1 Created 2.8s
✓ Container docker-ssrf_proxy-1 Created 2.9s
✓ Container docker-db_postgres-1 Healthy 7.8s
✓ Container docker-worker-1 Created 0.3s
✓ Container docker-api-1 Created 0.3s
✓ Container docker-worker_beat-1 Created 0.3s
✓ Container docker-plugin_daemon-1 Created 0.2s
✓ Container docker-nginx-1 Created 0.3s
pi@raspberrypi:~ $

```

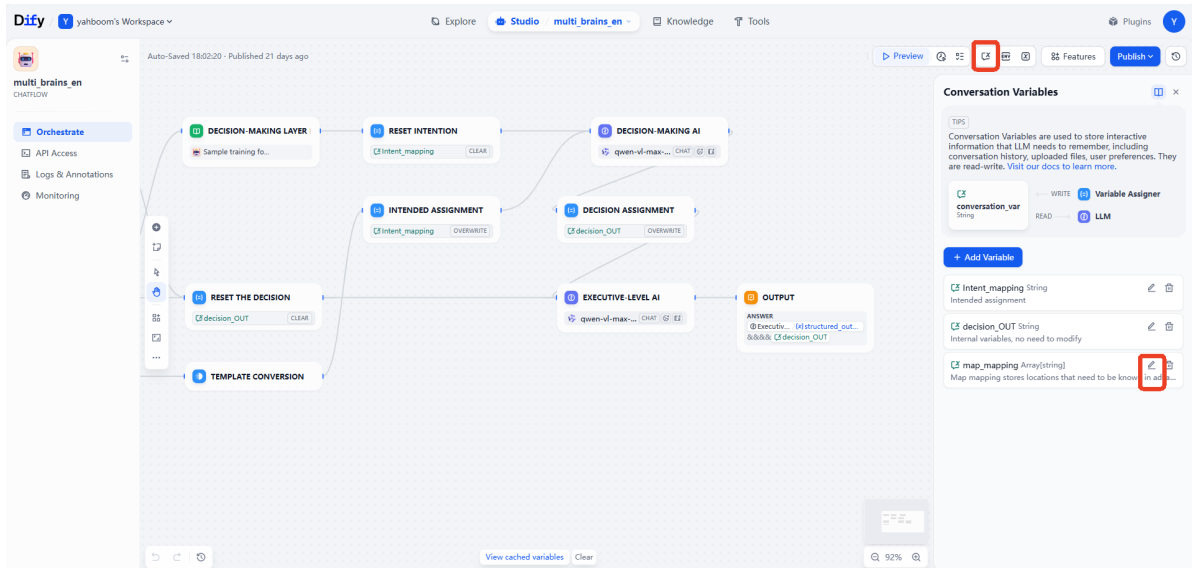
- Enter the vehicle's IP address directly into your browser's address bar to access the Dify management page, then click to select the corresponding AI application.

[!NOTE]

- Domestic users: multi_brains_zh
- International users: multi_brains_en



- Click to select **Session variables** in the upper right corner, and then click the edit button for the **map_mapping** variable.



- In the pop-up **Edit Session variables** window, edit the mapping relationships according to the settings in the previous map mapping file, and then click Save.

Edit Conversation Variable ×

Name
map_mapping

Type
array[string]

Default Value [Edit in JSON](#)

School: A

Library: B

+ Add Value

Description
Map mapping stores locations that need to be known in advance by AI

Cancel **Save**

Conversa

TIPS
Conversational information conversations are read-w

conversa
String

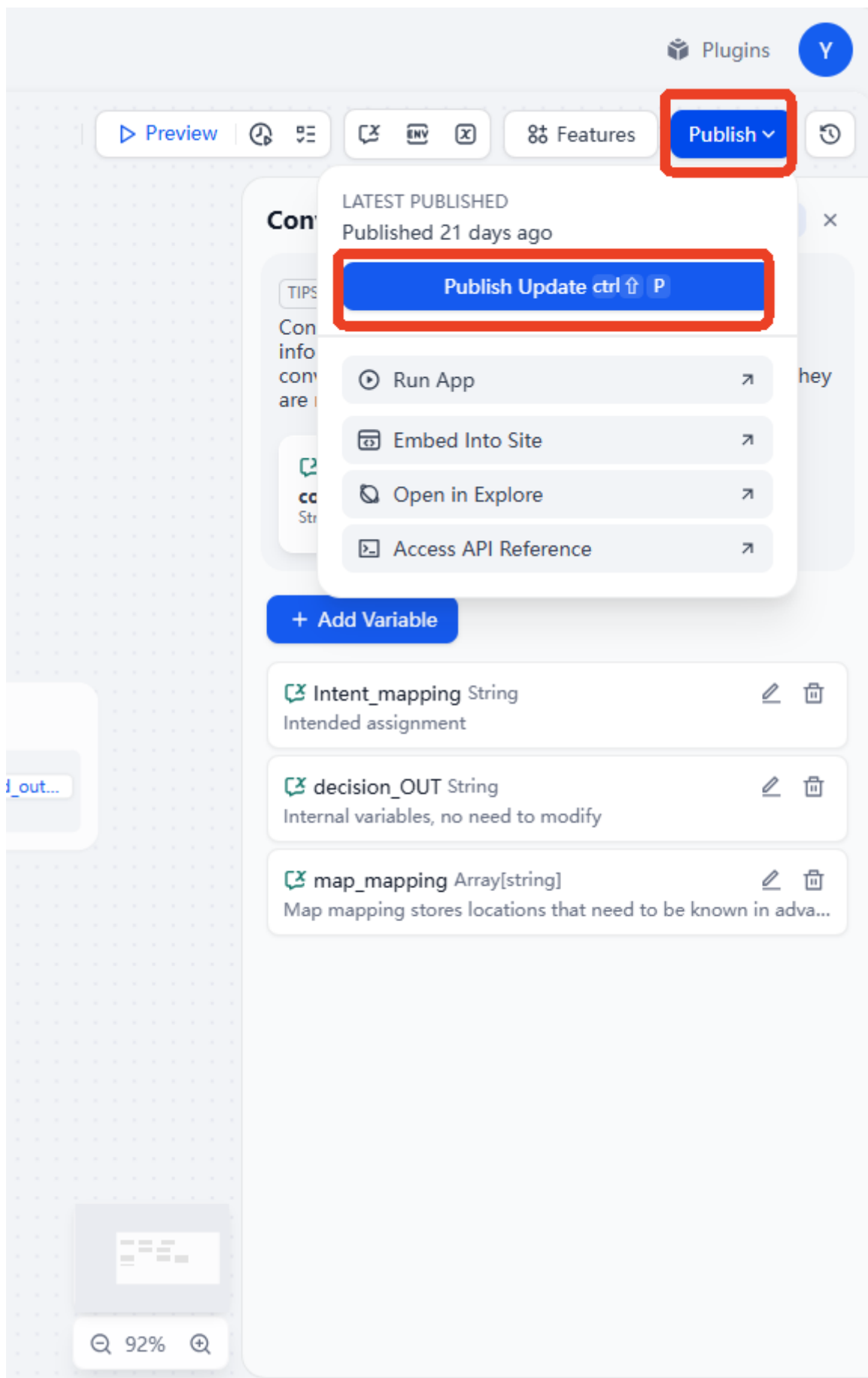
+ Add V

Intent_
Intended a

decisio
Internal var

map_m
Map mapp

Finally, remember to click "Publish" -> "Publish Update" to save your changes.



3. Running Examples

3.1 Starting the Program

- Open the terminal on the vehicle's infotainment system and enter the command to start the AI intelligent agent system, or you can use the shortcut: `multi_brains` to start the AI large model function:

```
ros2 launch m3_bringup llm_agent_control.launch.py
```

- Command to start navigation on the vehicle's infotainment system

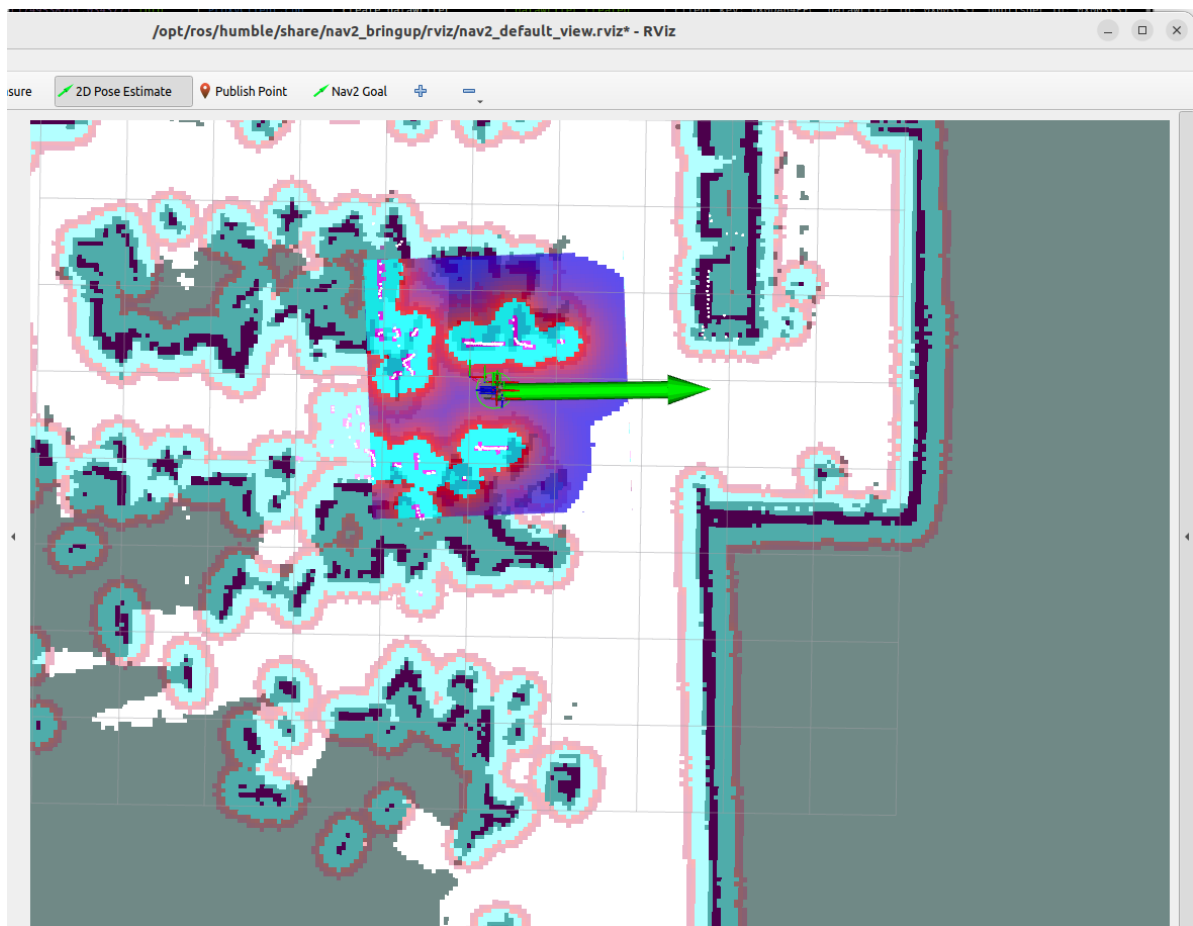
(ORIN & PI5 mainboard)

```
ros2 launch m3_bringup nav2.launch.py
```

(RDK mainboard) Run in steps, and visualize the process by starting the accompanying virtual machine `Rosmaster-M3-Ubuntu22.04`:

```
#Virtual Machine
rviz2 -d /home/yahboom/yahboomM3_ws/user_ws/src/m3_bringup/rviz/nav2.rviz
#Host side
ros2 launch m3_bringup nav2.launch.py gui:=False
```

- Afterwards, follow the navigation activation process to initialize positioning. This will open the rviz2 visualization interface. Click on **2D Pose Estimate** in the top toolbar to enter selection mode. Roughly mark the robot's location and orientation on the map. After initial positioning, the preparation is complete.



3.2 Test Cases

These test cases are for reference only. Provide instructions according to your own needs.

- Please remember your current location, then navigate to the kitchen and the master bedroom in that order, remembering the items you see in each room. Finally, return to your starting position and tell me what you saw in those two places.
- Wake up the robot and issue commands. The execution layer model will execute sub-tasks according to the task steps planned by the decision layer model. 1. Navigate to the "master bedroom" and observe the items in the environment. Place a pen in the simulated "master bedroom" and a pack of toilet paper in the simulated "kitchen":

```
jetson@yahboom: ~  
[asr-5] [INFO] [1750385414.028556399] [asr]: 0  
[asr-5] [INFO] [1750385414.119921759] [asr]: 0  
[asr-5] [INFO] [1750385414.210165105] [asr]: 0  
[asr-5] [INFO] [1750385414.300471621] [asr]: 0  
[asr-5] [INFO] [1750385414.390265068] [asr]: 0  
[asr-5] [INFO] [1750385414.481523321] [asr]: 0  
[asr-5] [INFO] [1750385414.571777356] [asr]: 0  
[asr-5] [INFO] [1750385414.662565773] [asr]: 0  
[asr-5] [INFO] [1750385414.754897462] [asr]: 0  
{'load_data': '0.028', 'extract_feat': '0.036', 'forward': '4.403', 'batch_size': '1', 'rtf': '0.263'},  
rtf_avg: 0.263: 100% | 1/1 [00:04<00:00, 4.40s/it]  
rtf_avg: 0.263: 100% | 1/1 [00:04<00:00, 4.40s/it]  
[asr-5] [INFO] [1750385419.211170831] [asr]: 请你先记住现在的位置然后依次导航去厨房和主卧室记住你所看到的  
物品最后返回出发时的位置告诉我你都在那两个地方看到了什么  
[asr-5] [INFO] [1750385419.211922083] [asr]: okay 😊, let me think for a moment...  
[model_service-3] [INFO] [1750385423.231742756] [model_service]: The upcoming task to be carried out: 1.  
调用记录当前位置函数记录位置2.导航去厨房, 调用获取当前视角图像函数获取图像并记录看到的物品3.导航去主卧室  
, 调用获取当前视角图像函数获取图像并记录看到的物品4.调用返回初始位置函数回到出发时的位置5.总结在厨房和主  
卧室看到的物品并告知用户  
[model_service-3] [INFO] [1750385427.315891765] [model_service]: "action": ['get_current_pose()'], "resp  
onse": 好的, 我现在就记住现在的位置, 就像记住了一个秘密基地一样。接下来, 我要去厨房和主卧室探险啦!  
[action_service-4] [INFO] [1750385436.995532898] [action_service]: Recorded Pose - Position: x=-0.070540  
17845186356, y=-0.07985647398462059, z=0.0  
[action_service-4] [INFO] [1750385436.996906727] [action_service]: Recorded Pose - Orientation: x=0.0, y  
=0.0, z=-0.3472589691926351, w=0.9377692724307342  
[model_service-3] [INFO] [1750385441.388327622] [model_service]: "action": ['navigation(G)'], "response"  
: 我已经记住了当前位置, 现在准备导航去厨房。就像一个勇敢的探险家, 出发!
```

- The robot executes the following steps according to the output flow of the decision layer:

```
[model_service-3] [INFO] [1750385441.388327622] [model_service]: "action": ['navigation(G)'], "response"  
: 我已经记住了当前位置, 现在准备导航去厨房。就像一个勇敢的探险家, 出发!  
[action_service-4] [INFO] [1750385449.596979417] [action_service]: navigation Goal accepted  
[action_service-4] [INFO] [1750385516.629335533] [action_service]: Navigation succeeded!  
[model_service-3] [INFO] [1750385520.996419381] [model_service]: "action": ['seewhat()'], "response": 我  
已经到达厨房了, 现在开始用我的火眼金睛观察这里有什么物品。  
[model_service-3] [INFO] [1750385537.403688769] [model_service]: "action": [], "response": 在厨房里, 我  
看到了一支笔静静地躺在地上。看来这里除了烹饪工具, 还藏着一些小秘密呢! 接下来, 我要去主卧室看看了。  
[model_service-3] [INFO] [1750385555.608727072] [model_service]: "action": ['navigation(A)'], "response"  
: 我已经记录了在厨房看到的物品, 现在准备前往主卧室。就像一个好奇的侦探, 继续我的探索之旅!  
[action_service-4] [INFO] [1750385565.962718690] [action_service]: navigation Goal accepted  
[action_service-4] [INFO] [1750385595.045950277] [action_service]: Navigation succeeded!  
[model_service-3] [INFO] [1750385599.609402332] [model_service]: "action": ['seewhat()'], "response": 我  
已经到达主卧室了, 现在开始仔细观察这里有什么物品。  
[model_service-3] [INFO] [1750385611.106135108] [model_service]: "action": [], "response": 在主卧室里,  
我看到了一盒橙色包装的纸巾放在光滑的地面上。这个地方看起来很整洁, 纸巾显得格外显眼。现在, 我要返回出发时  
的位置了。  
[model_service-3] [INFO] [1750385632.170785773] [model_service]: "action": ['navigation(zero)'], "respon  
se": 我已经记录了在主卧室看到的物品, 现在准备返回出发时的位置。就像完成了一次小旅行, 满载而归!  
[action_service-4] [INFO] [1750385642.582316949] [action_service]: navigation Goal accepted  
[action_service-4] [INFO] [1750385672.035012715] [action_service]: Navigation succeeded!  
[model_service-3] [INFO] [1750385679.513325513] [model_service]: "action": [], "response": 我已经回到了  
出发时的位置。让我来总结一下我在厨房和主卧室看到的物品吧! 在厨房, 我看到了一支笔静静地躺在地上; 而在主卧  
室, 我则发现了一盒橙色包装的纸巾放在光滑的地面上。这两个地方各有特色, 是不是很有趣呢?  
[model_service-3] [INFO] [1750385706.296938429] [model_service]: "action": ['finishtask()'], "response":  
我已经完成了全部任务, 告诉了你在厨房和主卧室看到的物品。如果还有其他需要帮忙的地方, 随时告诉我哦!
```

4. Source Code Analysis

- Robot motion source code path:

```
~/yahboomM3_ws/code_ws/src/multi_brains/multi_brains/action_service.py
```

action_service.py program:

Case 1 uses the **seewhat**, **navigation**, **load_target_points**, and **get_current_pose** methods from the **CustomActionServer** class. **Seewhat** has already been explained in the **Multimodal Visual Understanding + Robotic Arm Grasping** chapter. This lesson explains the newly introduced **navigation**, **load_target_points**, and **get_current_pose** functions.

init_ros_comunication initialization function:

- Creates a nav2 navigation client to request the ros2 navigation action server for subsequent navigation target point requests; creates a TF listener to monitor coordinate transformations between the map and basefootprint.

```
# Create a navigation client and request the navigation action server.
self.navclient=ActionClient(self, NavigateToPose, 'navigate_to_pose')
# Create a TensorFlow listener to monitor coordinate transformations
self.tf_buffer = Buffer()
self.tf_listener = TransformListener(self.tf_buffer, self)
```

The `load_target_points` function:

- Loads the target point coordinates from the map mapping file `map_mapping.yaml` and creates a navigation dictionary to store the characters and their corresponding map coordinates, where each point coordinate is a PoseStamped type data.

```
def load_target_points(self):
    """
    加载地图映射文件 /Load map mapping file
    """
    self.navpose_dict = {}
    self.road_net_dict = {}
    with open(self.map_mapping_file, "r") as file:
        full_target_points = yaml.safe_load(file)
        common_target_points = full_target_points.get("common_map_areas",
    {})

        road_net_target_points =
full_target_points.get("road_net_map_areas", {})

        for name, data in common_target_points.items():#加载常规导航点 / Load
common navigation points
            pose = PoseStamped()
            pose.header.frame_id = "map"
            pose.pose.position.x = data["position"]["x"]
            pose.pose.position.y = data["position"]["y"]
            pose.pose.position.z = data["position"]["z"]
            pose.pose.orientation.x = data["orientation"]["x"]
            pose.pose.orientation.y = data["orientation"]["y"]
            pose.pose.orientation.z = data["orientation"]["z"]
            pose.pose.orientation.w = data["orientation"]["w"]
            self.navpose_dict[name] = pose
```

The `__normal_navigation` function:

- Receives a character parameter (corresponding to the character in the map mapping above), parses the coordinates of the corresponding character from the dictionary, and uses the self.navclient navigation client object to request the ROS2 navigation action server. When the navigation action server returns a value of 4, it indicates successful navigation; other values indicate failure (possibly due to obstacles, planning failure, etc.). After navigation is complete, the result of the action execution is fed back to the large model.

```
def __normal_navigation(self, point_name)->None:
    '''常规导航功能 / Normal navigation function'''
    self.navigation_finish_flag = False
    self.goal_handle = None
    self.result = None
    self.res = None
    point_name = point_name.strip("\")
    if point_name not in self.navpose_dict:
        self.get_logger().error(f"Target point '{point_name}' does not exist
in the navigation dictionary.")
        return None

    if self.first_record:
        # 出发前记录当前在全局地图中的坐标(只有在每个任务周期的第一次执行时才会记录)/
before starting a new task, record the current pose in the global map
        transform = self.tf_buffer.lookup_transform(
            "map", "base_footprint", rclpy.time.Time()
        )
        pose = PoseStamped()
        pose.header.frame_id = "map"
        pose.pose.position.x = transform.transform.translation.x
        pose.pose.position.y = transform.transform.translation.y
        pose.pose.position.z = 0.0
        pose.pose.orientation = transform.transform.rotation
        self.navpose_dict["zero"] = pose
        self.road_net_dict["zero"] = pose
        self.first_record = False

    # 获取目标点坐标 /get_target_pose
    target_pose = self.navpose_dict.get(point_name)
    goal_msg = NavigateToPose.Goal()
    goal_msg.pose = target_pose
    send_goal_future = self.navclient.send_goal_async(goal_msg)

    def goal_response_callback(future):
        self.goal_handle = future.result()
        if not self.goal_handle or not self.goal_handle.accepted:
            self.get_logger().error("Goal was rejected!")
            return None

        get_result_future = self.goal_handle.get_result_async()

    def result_callback(future_result):
        self.result = future_result.result()
        self.navigation_finish_flag = True
        if self.result.status == 4:
            self.get_logger().info("Navigation finished!")
            self.res = True
        else:
```



```

        self.get_logger().info(f"Navigation failed with status:
{self.result.status}")
        self.res= False

        get_result_future.add_done_callback(result_callback)
        send_goal_future.add_done_callback(goal_response_callback)

    while not self.navigation_finish_flag:
        if self.interrupt_event.is_set() :
            self.navclient._cancel_goal(self.goal_handle)
            return None
        time.sleep(0.1)
    self.stop()
    return self.res

```

The `get_current_pose` function retrieves the robot's current map coordinates in the global coordinate system and adds these coordinates to a dictionary for later retrieval.

```

def get_current_pose(self):
    """
    获取当前在全局地图坐标系下的位置 /Get the current position in the global map
    coordinate system
    """
    transform = self.tf_buffer.lookup_transform("map", "base_footprint",
rclpy.time.Time())
    pose = PoseStamped()
    pose.header.frame_id = "map"
    pose.pose.position.x = transform.transform.translation.x
    pose.pose.position.y = transform.transform.translation.y
    pose.pose.position.z = 0.0
    pose.pose.orientation = transform.transform.rotation
    self.navpose_dict["zero"] = pose
    position = pose.pose.position
    orientation = pose.pose.orientation
    self.get_logger().info(f"Recorded Pose: Position: x={position.x}, y=
{position.y},z={position.z},Orientation: x={orientation.x}, y={orientation.y}, z=
{orientation.z}, w={orientation.w}")
    return True

```

